

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №2
на тему
«СИММЕТРИЧНАЯ КРИПТОГРАФИЯ. СТБ 34.101.31-2011»

БГУИР 6-05-0612-02 098

Выполнил студент группы 353502
ПЕТРОЧЕНКО Михаил Максимович

(дата, подпись студента)

Проверил ассистент кафедры информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Цель работы	3
2	Ход работы	4
2.1	Режим простой замены	5
2.2	Режим сцепления блоков	6
2.3	Режим счётчика	6
2.4	Режим гаммирования с обратной связью	7
	Заключение	8
	Приложение А (обязательное) Листинг программного кода	9
	Приложение Б (обязательное) Блок-схема алгоритма	18

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является изучение принципов построения симметричных блочных шифров на примере белорусского стандарта шифрования СТБ 34.101.31-2011, определяющего алгоритм BelT. Задачи включают изучение структуры блочного преобразования, положенного в основу стандарта, проектирование и реализацию программного средства, осуществляющего шифрование и дешифрование данных в режимах простой замены и гаммирования с обратной связью, а также генерацию имитовставки и тестирование разработанного программного средства.

Конечным результатом данной лабораторной работы должна быть разработанная программа, осуществляющая шифрование и дешифрование данных по СТБ 34.101.31-2011 во всех перечисленных режимах как для текста, вводимого с клавиатуры, так и для файлов произвольного содержания, и вычисляющая имитовставку заданной длины.

2 ХОД РАБОТЫ

В ходе выполнения работы было создано программное средство на языке программирования *Rust*. Реализация алгоритма BelT вынесена в отдельный модуль `encryption::bel_t`, взаимодействие с пользователем осуществляется через текстовое меню, позволяющее выбрать источник данных (текст или файл), выполняемую операцию (зашифрование, расшифрование или вычисление имитовставки), режим работы алгоритма и 256-битный ключ. Помимо предусмотренных заданием режимов простой замены и гаммирования с обратной связью дополнительно реализованы режимы сцепления блоков и счётчика, описанные тем же стандартом. Общий вид меню программы представлен на рисунке 2.1.

```
BelT (STB 34.101.31-2011) - input/output: text
1) encrypt (ECB)      5) decrypt (ECB)
2) encrypt (CBC)      6) decrypt (CBC)
3) encrypt (CTR)      7) decrypt (CTR)
4) encrypt (CFM)      8) decrypt (CFM)
9) MAC
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): k

Key is: 0000000100000002000000030000000400000005000000060000000700000008
```

Рисунок 2.1 – Меню программы

Алгоритм обрабатывает блок длиной 128 бит, представленный четырьмя 32-битными словами, и использует ключ длиной 256 бит, разделённый на восемь 32-битных подключей. Преобразование блока состоит из восьми тактов, в каждом из которых выполняется семь обращений к подключам, поэтому за полное преобразование каждое слово ключа используется семь раз. Единственным нелинейным элементом является таблица замены H , переводящая байт в байт; вспомогательная функция G_r применяет эту таблицу к каждому байту слова и циклически сдвигает результат влево на r бит, причём используются три величины сдвига: 5, 13 и 21. В отличие от сети Фейстеля за один такт изменяются все четыре слова блока, чем и объясняется малое число тактов. Номер такта участвует в вычислениях как раундовая константа, что исключает совпадение тактов между собой.

Зашифрование и расшифрование реализованы как независимые операции: при расшифровании такты выполняются в обратном порядке, подключи внутри такта распределяются зеркально, а итоговая перестановка слов отличается от применяемой при зашифровании. Блок-схема базового преобразования блока приведена в приложении Б.

Особенностью стандарта является отказ от дополнения сообщения до целого числа блоков. В режимах простой замены и сцепления блоков неполный последний блок обрабатывается приёмом кражи шифртекста, при котором недостающие байты заимствуются у зашифрованного предпоследнего блока, а в режимах гаммирования последний блок гаммы усекается по фактической

длине. Благодаря этому длина шифртекста совпадает с длиной открытого текста, а в режимах, использующих синхропосылку, превышает её ровно на 16 байт.

Перед расшифрованием выполняется проверка шифротекста на соответствие выбранному режиму: в режимах, использующих кражу шифртекста, длина данных не может быть меньше одного блока, а в режимах, использующих синхропосылку, шифротекст должен содержать шестнадцатибайтовую синхропосылку.

Корректность реализации подтверждена контрольными примерами приложения А стандарта: для базового преобразования блока, для каждого из четырёх режимов работы, включая случаи неполного последнего блока, и для имитовставки результаты совпадают с приведёнными в стандарте побитово. Проверка выполнена модульными тестами, входящими в состав программного средства.

Исходный код разработанного программного средства приведён в приложении А: реализация алгоритма представлена в листинге А.1, точка входа в программу – в листинге А.2.

2.1 Режим простой замены

В режиме простой замены каждый 128-битный блок открытого текста преобразуется независимо от остальных, что не требует синхропосылки, но сохраняет одинаковые блоки шифротекста для одинаковых блоков открытого текста. Результат работы программы в данном режиме представлен на рисунке 2.2: открытому тексту из повторяющихся символов соответствует шифротекст, в котором один и тот же шестнадцатибайтовый блок повторяется дважды. Длина шифротекста равна длине открытого текста, поскольку дополнение не применяется.

```
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 1

Plaintext: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
mode      : ECB (encryption)
plaintext : 32 bytes
encrypted : 4952B89AE073C92190C9BA9DAE1906054952B89AE073C92190C9BA9DAE190605 (32 bytes)

BeIT (STB 34.101.31-2011) - input/output: text
1) encrypt (ECB)      5) decrypt (ECB)
2) encrypt (CBC)      6) decrypt (CBC)
3) encrypt (CTR)      7) decrypt (CTR)
4) encrypt (CFM)      8) decrypt (CFM)
9) MAC
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 5

Ciphertext (hex): 4952B89AE073C92190C9BA9DAE1906054952B89AE073C92190C9BA9DAE190605
mode      : ECB (decryption)
encrypted : 32 bytes
decrypted : AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA (32 bytes)
```

Рисунок 2.2 – Зашифрование и расшифрование в режиме простой замены

2.2 Режим сцепления блоков

В режиме сцепления блоков каждый блок открытого текста перед зашифрованием складывается по модулю два с предыдущим блоком шифротекста, а роль нулевого блока играет синхропосылка. Благодаря этому одинаковым блокам открытого текста соответствуют различные блоки шифротекста, а зашифрование становится строго последовательным. Результат работы программы в данном режиме представлен на рисунке 2.3.

```
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 2
Plaintext: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
mode      : CBC (encryption)
plaintext : 32 bytes
encrypted : 7F08BC066F14F4B69C92C8F60920138FEE127A7FA3562433F5A04BB4C4FC251742140ACF294CE786BED8597F491625DA (48 bytes)

BeIT (STB 34.101.31-2011) - input/output: text
1) encrypt (ECB)      5) decrypt (ECB)
2) encrypt (CBC)      6) decrypt (CBC)
3) encrypt (CTR)      7) decrypt (CTR)
4) encrypt (CFM)      8) decrypt (CFM)
9) MAC
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 6

Ciphertext (hex): 7F08BC066F14F4B69C92C8F60920138FEE127A7FA3562433F5A04BB4C4FC251742140ACF294CE786BED8597F491625DA
mode      : CBC (decryption)
encrypted : 48 bytes
decrypted : AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA (32 bytes)
```

Рисунок 2.3 – Зашифрование и расшифрование в режиме сцепления блоков

2.3 Режим счётчика

В режиме счётчика шифрование выполняется наложением на открытый текст гаммы, вырабатываемой зашифрованием последовательных значений счётчика. Начальное значение счётчика получается зашифрованием синхропосылки, что исключает пересечение гаммы для близких синхропосылок. Синхропосылка вырабатывается случайным образом при каждом зашифровании и передаётся в начале шифротекста. Данный режим не требует дополнения сообщения до целого числа блоков и исключает совпадение блоков шифротекста. Результат работы программы в данном режиме представлен на рисунке 2.4.

```
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 3
Plaintext: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
mode      : CTR (encryption)
plaintext : 32 bytes
encrypted : 9BE5A9404A62B9DBD31B026F03C75443BA064450452A3494B913669E5A51199A6DF6F6D63B67421123CB69357BA14EBA (48 bytes)

BeIT (STB 34.101.31-2011) - input/output: text
1) encrypt (ECB)      5) decrypt (ECB)
2) encrypt (CBC)      6) decrypt (CBC)
3) encrypt (CTR)      7) decrypt (CTR)
4) encrypt (CFM)      8) decrypt (CFM)
9) MAC
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 7

Ciphertext (hex): 9BE5A9404A62B9DBD31B026F03C75443BA064450452A3494B913669E5A51199A6DF6F6D63B67421123CB69357BA14EBA
mode      : CTR (decryption)
encrypted : 48 bytes
decrypted : AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA (32 bytes)
```

Рисунок 2.4 – Зашифрование и расшифрование в режиме счётчика

2.4 Режим гаммирования с обратной связью

В режиме гаммирования с обратной связью первый блок гаммы вырабатывается зашифрованием синхропосылки, а каждый последующий – зашифрованием предыдущего блока шифротекста. Благодаря этому одинаковым блокам открытого текста соответствуют различные блоки шифротекста, а искажение одного блока влияет на результат расшифрования последующих. Блочное расшифрование в данном режиме, как и в режиме счётчика, не используется вовсе: гамма вырабатывается зашифрованием и при зашифровании, и при расшифровании. Результат работы программы в данном режиме представлен на рисунке 2.5.

```
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 4
Plaintext: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
mode      : CFM (encryption)
plaintext : 32 bytes
encrypted : EA5FF27E43FE71B716A7DE6D93A348751128DF0218A3415D447E22E42F16BCCCE51F921E921C616A00F004F1ADB06A57A (48 bytes)

BeIT (STB 34.101.31-2011) - input/output: text
1) encrypt (ECB)      5) decrypt (ECB)
2) encrypt (CBC)      6) decrypt (CBC)
3) encrypt (CTR)      7) decrypt (CTR)
4) encrypt (CFM)      8) decrypt (CFM)
9) MAC
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 8

Ciphertext (hex): EA5FF27E43FE71B716A7DE6D93A348751128DF0218A3415D447E22E42F16BCCCE51F921E921C616A00F004F1ADB06A57A
mode              : CFM (decryption)
encrypted         : 48 bytes
decrypted         : AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA (32 bytes)
```

Рисунок 2.5 – Зашифрование и расшифрование в режиме гаммирования с обратной связью

Таким образом все реализованные режимы работы описываемого стандартом СТБ 34.101.31-2011 алгоритма были изучены, реализованы и протестированы в полном объёме как для текста, вводимого с клавиатуры, так и для файлов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены принципы построения симметричных блочных шифров на примере алгоритма BelT, определённого стандартом СТБ 34.101.31-2011. Было разработано программное средство на языке *Rust*, реализующее шифрование и дешифрование данных в режимах простой замены и гаммирования с обратной связью, а также дополнительно в режимах сцепления блоков и счётчика, и генерацию имитовставки заданной длины. Зашифрование и расшифрование выполняются как отдельные операции над текстом, вводимым с клавиатуры, либо над файлами произвольного содержания.

Тестирование показало, что расшифрование во всех реализованных режимах восстанавливает исходное сообщение без искажений, а результаты преобразований совпадают с контрольными примерами приложения А стандарта, что подтверждает корректность реализации алгоритма. Сравнение режимов показало, что режим простой замены сохраняет структуру открытого текста и потому непригоден для шифрования сообщений, содержащих повторяющиеся блоки, тогда как остальные реализованные режимы лишены данного недостатка: в режиме гаммирования с обратной связью каждый блок гаммы вырабатывается зашифрованием предыдущего блока шифротекста, поэтому повторяющиеся блоки открытого текста не приводят к повторениям в шифротексте.

Отдельно следует отметить отличия рассмотренного стандарта от изученного ранее ГОСТ 28147-89: вдвое больший размер блока, фиксированная в тексте стандарта таблица замены, существенно меньшее число тактов преобразования и отказ от дополнения сообщения в пользу кражи шифротекста, благодаря которому длина сообщения сохраняется во всех режимах.

Все задачи лабораторной работы выполнены в полном объёме.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

Листинг А.1 – Реализация алгоритма BelT на *Rust*

```
use std::fmt::Display;

use common::key::Key;
use rand::random;

const BLOCK_SIZE: usize = 16;
const ITERATIONS: usize = 8;
pub const MAX_MAC_BITS: usize = 64;
const H_TABLE: [u8; 256] = [
    0xB1, 0x94, 0xBA, 0xC8, 0x0A, 0x08, 0xF5, 0x3B, 0x36, 0x6D, 0x00, 0x8E,
    0x58, 0x4A, 0x5D, 0xE4,
    0x85, 0x04, 0xFA, 0x9D, 0x1B, 0xB6, 0xC7, 0xAC, 0x25, 0x2E, 0x72, 0xC2,
    0x02, 0xFD, 0xCE, 0x0D,
    0x5B, 0xE3, 0xD6, 0x12, 0x17, 0xB9, 0x61, 0x81, 0xFE, 0x67, 0x86, 0xAD,
    0x71, 0x6B, 0x89, 0x0B,
    0x5C, 0xB0, 0xC0, 0xFF, 0x33, 0xC3, 0x56, 0xB8, 0x35, 0xC4, 0x05, 0xAE,
    0xD8, 0xE0, 0x7F, 0x99,
    0xE1, 0x2B, 0xDC, 0x1A, 0xE2, 0x82, 0x57, 0xEC, 0x70, 0x3F, 0xCC, 0xF0,
    0x95, 0xEE, 0x8D, 0xF1,
    0xC1, 0xAB, 0x76, 0x38, 0x9F, 0xE6, 0x78, 0xCA, 0xF7, 0xC6, 0xF8, 0x60,
    0xD5, 0xBB, 0x9C, 0x4F,
    0xF3, 0x3C, 0x65, 0x7B, 0x63, 0x7C, 0x30, 0x6A, 0xDD, 0x4E, 0xA7, 0x79,
    0x9E, 0xB2, 0x3D, 0x31,
    0x3E, 0x98, 0xB5, 0x6E, 0x27, 0xD3, 0xBC, 0xCF, 0x59, 0x1E, 0x18, 0x1F,
    0x4C, 0x5A, 0xB7, 0x93,
    0xE9, 0xDE, 0xE7, 0x2C, 0x8F, 0x0C, 0x0F, 0xA6, 0x2D, 0xDB, 0x49, 0xF4,
    0x6F, 0x73, 0x96, 0x47,
    0x06, 0x07, 0x53, 0x16, 0xED, 0x24, 0x7A, 0x37, 0x39, 0xCB, 0xA3, 0x83,
    0x03, 0xA9, 0x8B, 0xF6,
    0x92, 0xBD, 0x9B, 0x1C, 0xE5, 0xD1, 0x41, 0x01, 0x54, 0x45, 0xFB, 0xC9,
    0x5E, 0x4D, 0x0E, 0xF2,
    0x68, 0x20, 0x80, 0xAA, 0x22, 0x7D, 0x64, 0x2F, 0x26, 0x87, 0xF9, 0x34,
    0x90, 0x40, 0x55, 0x11,
    0xBE, 0x32, 0x97, 0x13, 0x43, 0xFC, 0x9A, 0x48, 0xA0, 0x2A, 0x88, 0x5F,
    0x19, 0x4B, 0x09, 0xA1,
    0x7E, 0xCD, 0xA4, 0xD0, 0x15, 0x44, 0xAF, 0x8C, 0xA5, 0x84, 0x50, 0xBF,
    0x66, 0xD2, 0xE8, 0x8A,
    0xA2, 0xD7, 0x46, 0x52, 0x42, 0xA8, 0xDF, 0xB3, 0x69, 0x74, 0xC5, 0x51,
    0xEB, 0x23, 0x29, 0x21,
    0xD4, 0xEF, 0xD9, 0xB4, 0x3A, 0x62, 0x28, 0x75, 0x91, 0x14, 0x10, 0xEA,
    0x77, 0x6C, 0xDA, 0x1D,
];

#[derive(Clone, Copy, PartialEq, Eq)]
pub enum BelTType {
    ECB,
    CBC,
    CTR,
    CFM,
}

impl Display for BelTType {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        let name = match self {
            BelTType::ECB => "ECB",
            BelTType::CBC => "CBC",
        }
    }
}
```

```

        BelTType::CTR => "CTR",
        BelTType::CFM => "CFM",
    };

    return write!(f, "{name}");
}

pub struct BelT {}

impl BelT {
    fn split_u128_le(n: u128) -> [u32; 4] {
        return [
            n as u32,
            (n >> 32) as u32,
            (n >> 64) as u32,
            (n >> 96) as u32,
        ];
    }

    fn g(r: u32, u: u32) -> u32 {
        return u32::from_le_bytes(u.to_le_bytes().map(|byte| H_TABLE[byte as
usize]))
            .rotate_left(r);
    }

    fn get_subkey(key: &Key<8>, idx: u32) -> u32 {
        return key[(idx % 8) as usize];
    }

    fn encrypt_block(block: u128, key: &Key<8>) -> u128 {
        let [mut a, mut b, mut c, mut d] = Self::split_u128_le(block);
        for iteration in 0..ITERATIONS {
            let i = (iteration + 1) as u32;
            b = b ^ Self::g(5, a.wrapping_add(Self::get_subkey(key, (7 * i -
6) - 1)));
            c = c ^ Self::g(21, d.wrapping_add(Self::get_subkey(key, (7 * i -
5) - 1)));
            a = a.wrapping_sub(Self::g(
                13,
                b.wrapping_add(Self::get_subkey(key, (7 * i - 4) - 1)),
            ));
            let e = i ^ Self::g(
                21,
                b.wrapping_add(c)
                    .wrapping_add(Self::get_subkey(key, (7 * i - 3) - 1)),
            );
            b = b.wrapping_add(e);
            c = c.wrapping_sub(e);
            d = d.wrapping_add(Self::g(
                13,
                c.wrapping_add(Self::get_subkey(key, (7 * i - 2) - 1)),
            ));
            b = b ^ Self::g(21, a.wrapping_add(Self::get_subkey(key, (7 * i -
1) - 1)));
            c = c ^ Self::g(5, d.wrapping_add(Self::get_subkey(key, (7 * i) -
1)));
            (a, b) = (b, a);
            (c, d) = (d, c);
            (b, c) = (c, b);
        }

        return (c as u128) << 96 | (a as u128) << 64 | (d as u128) << 32 | (b

```

```

as u128);
    }

    fn decrypt_block(block: u128, key: &Key<8>) -> u128 {
        let [mut a, mut b, mut c, mut d] = Self::split_u128_le(block);

        for iteration in (0..ITERATIONS).rev() {
            let i = (iteration + 1) as u32;
            b = b ^ Self::g(5, a.wrapping_add(Self::get_subkey(key, (7 * i) -
1)))));
            c = c ^ Self::g(21, d.wrapping_add(Self::get_subkey(key, (7 * i -
1) - 1)))));
            a = a.wrapping_sub(Self::g(
                13,
                b.wrapping_add(Self::get_subkey(key, (7 * i - 2) - 1)),
            ));
            let e = i ^ Self::g(
                21,
                b.wrapping_add(c)
                    .wrapping_add(Self::get_subkey(key, (7 * i - 3) - 1)),
            );
            b = b.wrapping_add(e);
            c = c.wrapping_sub(e);
            d = d.wrapping_add(Self::g(
                13,
                c.wrapping_add(Self::get_subkey(key, (7 * i - 4) - 1)),
            ));
            b = b ^ Self::g(21, a.wrapping_add(Self::get_subkey(key, (7 * i -
5) - 1)))));
            c = c ^ Self::g(5, d.wrapping_add(Self::get_subkey(key, (7 * i -
6) - 1)))));
            (a, b) = (b, a);
            (c, d) = (d, c);
            (a, d) = (d, a);
        }

        return (b as u128) << 96 | (d as u128) << 64 | (a as u128) << 32 | (c
as u128);
    }

    fn block_from(bytes: &[u8]) -> u128 {
        return u128::from_le_bytes(
            bytes
                .try_into()
                .expect("u128 can only be constructed from [u8;16]"),
        );
    }

    fn apply_ecb(bytes: Vec<u8>, key: &Key<8>, encrypt: bool) -> Vec<u8> {
        assert!(
            bytes.len() >= BLOCK_SIZE,
            "belt-ecb needs at least one whole block"
        );

        let full = bytes.len() / BLOCK_SIZE;
        let rest = bytes.len() % BLOCK_SIZE;

        let apply = |n: u128| {
            if encrypt {
                Self::encrypt_block(n, key)
            } else {
                Self::decrypt_block(n, key)
            }
        }

```

```

};

let mut result: Vec<u8> = Vec::with_capacity(bytes.len());

for (idx, block) in bytes.chunks(BLOCK_SIZE).enumerate() {
    if idx == full {
        break;
    }

    let n = Self::block_from(block);
    let processed = apply(n);
    result.extend_from_slice(&processed.to_le_bytes());
}

if rest != 0 {
    let second_to_last_block_start = (full - 1) * BLOCK_SIZE;
    let last_block_start = full * BLOCK_SIZE;
    result.resize(result.len() + rest, 0);

    result[last_block_start..].copy_from_slice(&bytes[last_block_start..]);
    for j in 0..rest {
        result.swap(last_block_start + j, second_to_last_block_start
+ j);
    }
    let processed = apply(Self::block_from(
        &result[second_to_last_block_start..last_block_start],
    ));
    result[second_to_last_block_start..last_block_start]
        .copy_from_slice(&processed.to_le_bytes());
}

return result;
}

pub fn encrypt_ecb(plain_bytes: Vec<u8>, key: &Key<8>) -> Vec<u8> {
    return Self::apply_ecb(plain_bytes, key, true);
}

pub fn decrypt_ecb(encrypted_bytes: Vec<u8>, key: &Key<8>) -> Vec<u8> {
    return Self::apply_ecb(encrypted_bytes, key, false);
}

fn apply_cbc(bytes: &[u8], key: &Key<8>, sync: u128, encrypt: bool) ->
Vec<u8> {
    assert!(
        bytes.len() >= BLOCK_SIZE,
        "belt-cbc needs at least one whole block"
    );

    let full = bytes.len() / BLOCK_SIZE;
    let rest = bytes.len() % BLOCK_SIZE;
    let chained = if rest == 0 { full } else { full - 1 };

    let apply = |n: u128| {
        if encrypt {
            Self::encrypt_block(n, key)
        } else {
            Self::decrypt_block(n, key)
        }
    };

    let mut prev: u128 = sync;
    let mut result: Vec<u8> = Vec::with_capacity(bytes.len());

```

```

    for block in bytes.chunks_exact(BLOCK_SIZE).take(chained) {
        let x = Self::block_from(block);
        let y = if encrypt {
            apply(x ^ prev)
        } else {
            apply(x) ^ prev
        };
        result.extend_from_slice(&y.to_le_bytes());

        prev = if encrypt { y } else { x };
    }

    if rest == 0 {
        return result;
    }

    let second_to_last_block_start = (full - 1) * BLOCK_SIZE;
    let last_block_start = full * BLOCK_SIZE;

    let x_n_m_1 =
Self::block_from(&bytes[second_to_last_block_start..last_block_start]);

    let mut x_n_buf = [0u8; BLOCK_SIZE];
    x_n_buf[..rest].copy_from_slice(&bytes[last_block_start..]);
    let x_n = Self::block_from(&x_n_buf);

    let y_n_r = if encrypt {
        apply(x_n_m_1 ^ prev)
    } else {
        apply(x_n_m_1) ^ x_n
    };

    let rest_bits = rest * 8;
    let y_n = y_n_r & (u128::MAX >> (128 - rest_bits));
    let r = y_n_r >> rest_bits;

    let combined = if encrypt {
        (x_n ^ y_n) | (r << rest_bits)
    } else {
        x_n | (r << rest_bits)
    };

    let y_n_m_1 = if encrypt {
        apply(combined)
    } else {
        apply(combined) ^ prev
    };

    result.extend_from_slice(&y_n_m_1.to_le_bytes());
    result.extend_from_slice(&y_n.to_le_bytes()[..rest]);

    return result;
}

pub fn encrypt_cbc(plain_bytes: Vec<u8>, key: &Key<8>) -> Vec<u8> {
    let sync: u128 = random();

    let mut result: Vec<u8> = Vec::with_capacity(BLOCK_SIZE +
plain_bytes.len());
    result.extend_from_slice(&sync.to_le_bytes());
    result.extend_from_slice(&Self::apply_cbc(&plain_bytes, key, sync,
true));
}

```

```

        return result;
    }

    pub fn decrypt_cbc(encrypted_bytes: Vec<u8>, key: &Key<8>) -> Vec<u8> {
        let (sync, body) = encrypted_bytes.split_at(BLOCK_SIZE);

        return Self::apply_cbc(body, key, Self::block_from(sync), false);
    }

    fn l(bits: usize, n: u128) -> u128 {
        return n & (u128::MAX >> (128 - bits));
    }

    fn apply_ctr(bytes: &[u8], key: &Key<8>, sync: u128) -> Vec<u8> {
        let full = bytes.len() / BLOCK_SIZE;
        let rest = bytes.len() % BLOCK_SIZE;

        let apply = |n: u128| Self::encrypt_block(n, key);

        let mut s: u128 = apply(sync);
        let mut result: Vec<u8> = Vec::with_capacity(bytes.len());

        for block in bytes.chunks_exact(BLOCK_SIZE) {
            s = s.wrapping_add(1);
            let x = Self::block_from(block);
            let y = x ^ apply(s);
            result.extend_from_slice(&y.to_le_bytes());
        }

        if rest != 0 {
            let last_block_start = full * BLOCK_SIZE;
            let mut x_n_buf = [0u8; BLOCK_SIZE];
            x_n_buf[..rest].copy_from_slice(&bytes[last_block_start..]);
            let x_n = Self::block_from(&x_n_buf);

            s = s.wrapping_add(1);

            let processed = x_n ^ Self::l(rest * 8, apply(s));

            result.extend_from_slice(&processed.to_le_bytes()[..rest]);
        }

        return result;
    }

    pub fn encrypt_ctr(plain_bytes: Vec<u8>, key: &Key<8>) -> Vec<u8> {
        let sync: u128 = random();

        let mut result: Vec<u8> = Vec::with_capacity(BLOCK_SIZE +
plain_bytes.len());
        result.extend_from_slice(&sync.to_le_bytes());
        result.extend_from_slice(&Self::apply_ctr(&plain_bytes, key, sync));

        return result;
    }

    pub fn decrypt_ctr(encrypted_bytes: Vec<u8>, key: &Key<8>) -> Vec<u8> {
        let (sync, body) = encrypted_bytes.split_at(BLOCK_SIZE);

        return Self::apply_ctr(body, key, Self::block_from(sync));
    }

```

```

    fn apply_cfm(bytes: &[u8], key: &Key<8>, sync: u128, encrypt: bool) ->
Vec<u8> {
    let full = bytes.len() / BLOCK_SIZE;
    let rest = bytes.len() % BLOCK_SIZE;

    let apply = |n: u128| Self::encrypt_block(n, key);

    let mut prev: u128 = sync;
    let mut result: Vec<u8> = Vec::with_capacity(bytes.len());

    for block in bytes.chunks_exact(BLOCK_SIZE) {
        let x = Self::block_from(block);
        let y = x ^ apply(prev);
        result.extend_from_slice(&y.to_le_bytes());
        prev = if encrypt { y } else { x };
    }

    if rest != 0 {
        let last_block_start = full * BLOCK_SIZE;
        let mut x_n_buf = [0u8; BLOCK_SIZE];
        x_n_buf[..rest].copy_from_slice(&bytes[last_block_start..]);
        let x_n = Self::block_from(&x_n_buf);
        let processed = x_n ^ Self::l(rest * 8, apply(prev));

        result.extend_from_slice(&processed.to_le_bytes()[..rest]);
    }

    return result;
}

pub fn encrypt_cfm(plain_bytes: Vec<u8>, key: &Key<8>) -> Vec<u8> {
    let sync: u128 = random();

    let mut result: Vec<u8> = Vec::with_capacity(BLOCK_SIZE +
plain_bytes.len());
    result.extend_from_slice(&sync.to_le_bytes());
    result.extend_from_slice(&Self::apply_cfm(&plain_bytes, key, sync,
true));

    return result;
}

pub fn decrypt_cfm(encrypted_bytes: Vec<u8>, key: &Key<8>) -> Vec<u8> {
    let (sync, body) = encrypted_bytes.split_at(BLOCK_SIZE);

    return Self::apply_cfm(body, key, Self::block_from(sync), false);
}

fn phi_1(u: u128) -> u128 {
    let (u1, u2, u3, u4) = (
        u as u32,
        (u >> 32) as u32,
        (u >> 64) as u32,
        (u >> 96) as u32,
    );

    return ((u1 ^ u2) as u128) << 96 | (u4 as u128) << 64 | (u3 as u128)
<< 32 | u2 as u128;
}

fn phi_2(u: u128) -> u128 {
    let (u1, u2, u3, u4) = (
        u as u32,

```

```

        (u >> 32) as u32,
        (u >> 64) as u32,
        (u >> 96) as u32,
    );

    return (u3 as u128) << 96 | (u2 as u128) << 64 | (u1 as u128) << 32 |
(u1 ^ u4) as u128;
}

fn psi(u: u128, word_len_bits: usize) -> u128 {
    return u | (1 << (word_len_bits + 7));
}

fn truncate_mac(s: u128, mac_bits: usize) -> u64 {
    let tag = u64::from_be_bytes(
        s.to_le_bytes()[..8]
        .try_into()
        .expect("a block always has 16 bytes"),
    );

    return tag >> (64 - mac_bits);
}

pub fn compute_mac(message: Vec<u8>, key: &Key<8>, mac_bits: usize) ->
Option<u64> {
    assert!(
        (1..=MAX_MAC_BITS).contains(&mac_bits),
        "mac bits must be in 1..={MAX_MAC_BITS}, got {mac_bits}"
    );

    let whole_tail = !message.is_empty() && message.len() % BLOCK_SIZE ==
0;
    let head_blocks = message.len() / BLOCK_SIZE - whole_tail as usize;

    let apply = |n: u128| Self::encrypt_block(n, key);
    let mut s = 0u128;
    let r = apply(s);

    for block in message.chunks_exact(BLOCK_SIZE).take(head_blocks) {
        s = apply(s ^ Self::block_from(block));
    }

    let tail = &message[head_blocks * BLOCK_SIZE..];
    let mut x_n_buf = [0u8; BLOCK_SIZE];
    x_n_buf[..tail.len()].copy_from_slice(tail);
    let x_n = Self::block_from(&x_n_buf);

    s = if whole_tail {
        s ^ x_n ^ Self::phi_1(r)
    } else {
        s ^ Self::psi(x_n, tail.len() * 8) ^ Self::phi_2(r)
    };

    return Some(Self::truncate_mac(apply(s), mac_bits));
}
}

```

Листинг А.2 – Точка входа в программу

```

mod encryption;
use common::cli::{self, CipherApp, CipherFn, CipherMode, MacSpec};
use common::key::Key;
use encryption::bel_t::*;

```

```

const KEY: Key<8> = Key([1, 2, 3, 4, 5, 6, 7, 8]);
const BLOCK_SIZE: usize = 16;
const DEFAULT_MAC_BITS: usize = 64;
const ENCRYPTED_EXTENSION: &str = "belt";

fn cipher_fns(encryption_type: BelTType) -> (CipherFn, CipherFn) {
    return match encryption_type {
        BelTType::ECB => (BelT::encrypt_ecb, BelT::decrypt_ecb),
        BelTType::CBC => (BelT::encrypt_cbc, BelT::decrypt_cbc),
        BelTType::CTR => (BelT::encrypt_ctr, BelT::decrypt_ctr),
        BelTType::CFM => (BelT::encrypt_cfm, BelT::decrypt_cfm),
    };
}

fn modes() -> Vec<CipherMode> {
    return [BelTType::ECB, BelTType::CBC, BelTType::CTR, BelTType::CFM]
        .into_iter()
        .map(|encryption_type| {
            let (encrypt, decrypt) = cipher_fns(encryption_type);
            let mode = CipherMode::new(encryption_type.to_string(), encrypt,
decrypt);

            return match encryption_type {
                BelTType::ECB => mode
                    .checking_plaintext(cli::at_least(BLOCK_SIZE))
                    .checking_ciphertext(cli::at_least(BLOCK_SIZE)),
                BelTType::CBC => mode
                    .checking_plaintext(cli::at_least(BLOCK_SIZE))
                    .checking_ciphertext(cli::at_least(2 * BLOCK_SIZE)),
                _ => mode.checking_ciphertext(cli::at_least(BLOCK_SIZE)),
            };
        })
        .collect();
}

fn main() {
    CipherApp {
        title: "BelT (STB 34.101.31-2011)",
        encrypted_extension: ENCRYPTED_EXTENSION,
        modes: modes(),
        mac: Some(MacSpec {
            compute: BelT::compute_mac,
            default_bits: DEFAULT_MAC_BITS,
            max_bits: MAX_MAC_BITS,
        }),
        digest: None,
        default_key: KEY,
    }
    .run();
}

```

ПРИЛОЖЕНИЕ Б **(обязательное)** **Блок-схема алгоритма**

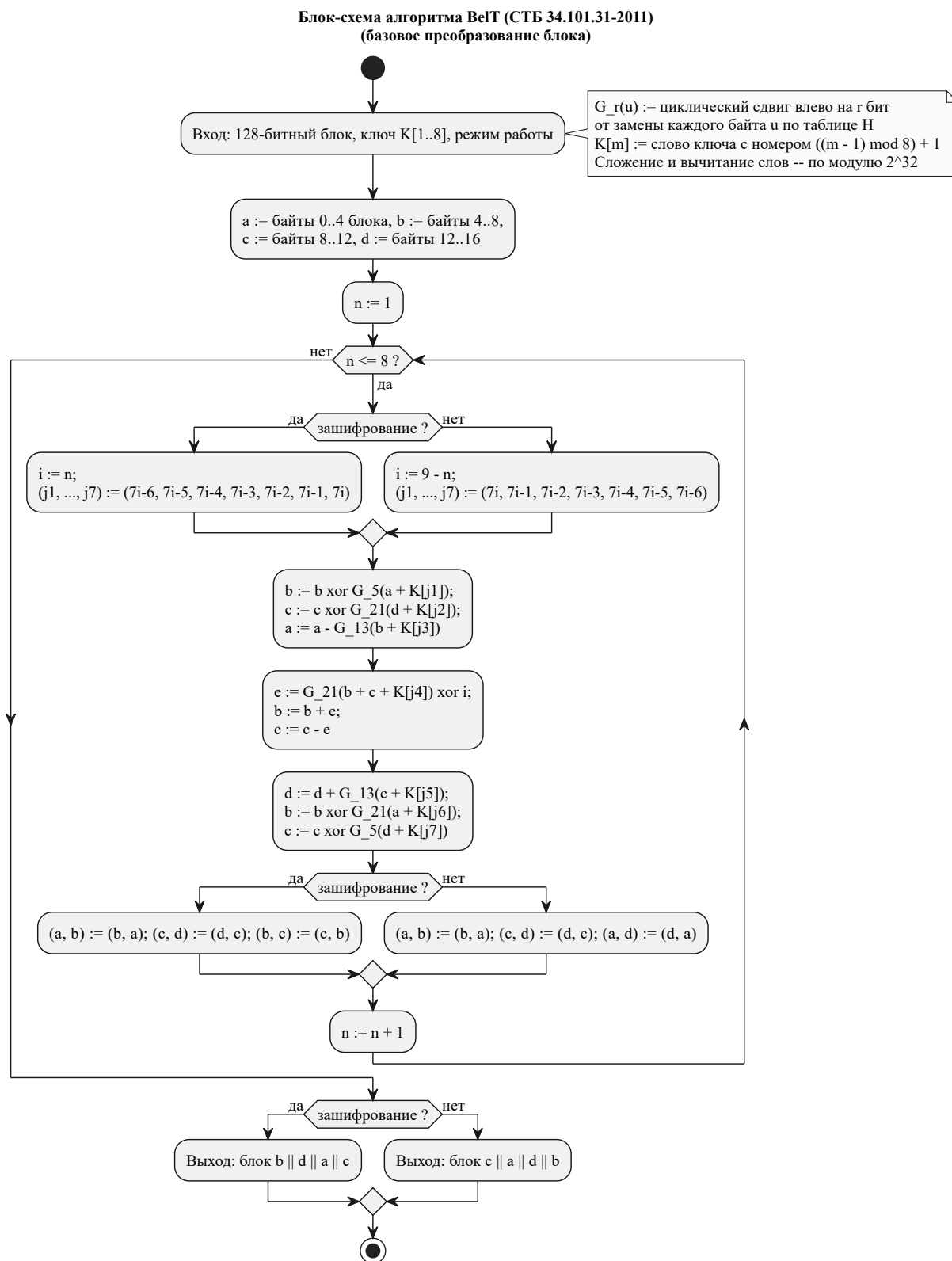


Рисунок Б.1 – Блок-схема алгоритма